

Реалізувати подані задачі, використовуючи структуру даних стек (в тому числі реалізувати контроль і захист стеку)

Записки і замальовки спостережень про життя стеків:



public

private (without inheritance)

```

init():          i ← 0
push(value):     i ← i+1; stack[i] ← value    // no overflow (limit ?)
pop():          if i>0 then return { stack[i]; i ← i-1 }
size():         return i
top():          if i>0 then return stack[i]
clear():        i ← 0
  
```

controlled

// все пропало! ☹

Рекомендуємо переглянути розширені виклади про рекурсію і стек, які тісно зв'язані:

<https://uk.javascript.info/recursion>

[https://uk.wikipedia.org/wiki/Рекурсія_\(програмування\)](https://uk.wikipedia.org/wiki/Рекурсія_(програмування))

https://uk.wikipedia.org/wiki/Взаємна_рекурсія

! До розв'язування окремих задач накладаємо певні умови. Недотримання умов вважаємо неправильним розв'язком задачі.

- 1) Алгоритми розв'язку задач в загальному випадку мають бути *нерекурсивні*.
- 2) Реалізувати *власний клас стеку*, включаючи контроль операцій і захист даних (неможливість зовнішнього втручання). Модуль `collections` чи подібні не використовувати. Моделювати стек за допомогою списку.
- 2) Для задач 1.1, 1.2, 1.3 *не використовувати* оператори чи функції сортування – прямі чи непрямі, явні чи неявні. (Python може неявно використовувати сортування при виконанні методів структур даних і функцій).

Задача 1.1.

Задано текст алгебраїчної формули в лінійній формі запису, який включає круглі дужки. Вважати, що текст складений коректно, тобто, до кожної відкриваючої дужки є закриваюча і навпаки.

Надрукувати попарно позиції відповідних відкриваючих і закриваючих дужок в довільному порядку розташування пар дужок.

Приклад вхідних даних в фільтрованому вигляді – перший рядок:

```
... (..... (.....) ..... (.....) ..... ( (..... (.....) ) .....) .....) ...
      3         6         10         15    19         21 22         27    29 30         33    36
```

Другий рядок позначає позиції дужок – для прикладу.

Задача 1.2. За умовою попередньої задачі надрукувати позиції пар дужок в порядку зростання номерів позицій відкриваючих дужок.

Задача 1.3. За умовою попередньої задачі надрукувати позиції пар дужок в порядку зростання номерів позицій закриваючих дужок.

Задача 2. Обчислити суму всіх елементів структури, яка складається з довільно вкладених списків чисел. Наприклад, для такої структури:

```
tree = [ 5, 1, [3, [2, 2], ], 6, [ [4, [7, 7], [3, 3, 3]] ] ]
```

Очевидний розв'язок задачі – рекурсивна функція (а чи очевидний ? ☺). Проте, як зазначено на початку, *не використовувати рекурсію пряму чи непряму*.

Рекомендація: складіть для вправи і перевірте спочатку рекурсивну функцію.

Задача 3. (Умов не накладаємо). В текстовому файлі чи в текстовому рядку записана формула, яка б мала відповідати таким правилам:

```
формула ::= терм ( ( + | - ) терм ) *
терм ::= ( x | y | z ) | "(" формула ")" | "[" формула "]"
        | "{" формула "}"
```

Наприклад: (правильно)

```
x + ( y - z - [ x + x ] + { [ z - z - y ] + ( y ) } ) - z
```

Перевірити, чи відповідає правилам запис довільно заданої формули.

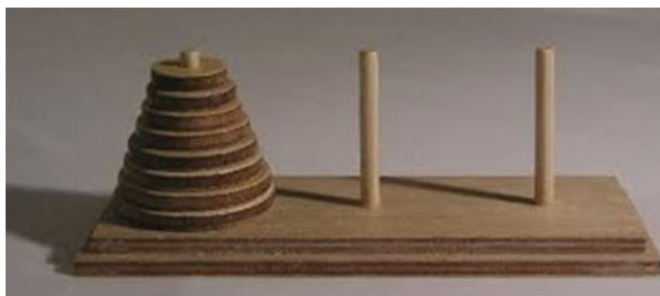
Записати як коментар на початку програми ідею та алгоритм розв'язку.

Задача 4. (Умов не накладаємо). В текстовому файлі чи в текстовому рядку записана без помилок формула за такими правилами:

```
формула ::= цифра | "S(" формула "," формула )" "
           | "D(" формула "," формула )" "
цифра ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

де S означає функцію суми двох аргументів, а D – функцію ділення. Обчислити як ціле число значення заданої формули. Наприклад, для D(8,S(2,1)) відповідь буде 2.

Задача 5.1. Давно відома задача про Ханойські вежі:



https://uk.wikipedia.org/wiki/Ханойська_вежа

Пригадати зміст задачі і правила. Класична задача – перенести вежу на інший стрижень. Стрижнів є лише три. Стрижень – це стек. Перенесення дисків – це методи (функції) `push()` і `pop()`. Пригадати алгоритм розв'язування.

- 1) Змодельовати програмно розв'язок задачі, використовуючи стеки. Як продемонструвати послідовні кроки розв'язку в розумній кількості, щоб можна було побачити і оцінити правильність? (частинна демонстрація, візуалізація).
- 2) Додати до розв'язку оператори, які б обчислювали кількість операцій перенесення дисків. Показати результати для $n=1,2,\dots,14$, де n – висота вежі (число дисків).

***Задача 5.2.** Подібна практична задача. На початку диски (або інші елементи чи деталі) складені на стрижень (контейнер) в довільному порядку, а не в сортованому. Треба перенести диски (деталі) на інший чи той самий стрижень (контейнер) так, щоб вони були сортовані. (Порядок підсумкового сортування не має значення, бо він легко міняється на протилежний одним циклом перенесення).

*Запропонувати і описати алгоритм розв'язку. Чи можна розв'язати таку задачу за допомогою лише трьох стрижнів? Які інші додаткові функції можуть бути корисними, крім `push()` і `pop()`, але не руйнуючи при цьому правила роботи стеку?

[Модель подібної інженерної задачі – алгоритми промислових пристроїв-автоматів]

Окремі вимоги до виконання всіх завдань

- 1) Реалізувати власний *єдиний клас стеку*, зовнішній і доступний до всіх функцій, і на його основі виконати програмування завдань. Стек має бути *захисний і контрольований*.
- 2) Скласти *тести* для перевірки кожної задачі в повному обсязі, включаючи перевірки граничних випадків. Передбачити зручний спосіб вибору тесту для виконання програми - бажано без редагування коду програми або з мінімальним редагуванням. В останньому випадку запишіть коментар чи окрему інструкцію, як це зробити. Тести мають бути частиною програмного коду в тому самому файлі.
- 3) Розв'язок кожної задачі (разом з тестами) оформити *окремим файлом*. Отже, може бути разом до 8 файлів.

Файли надіслати окремими вкладеннями, НЕ АРХІВУВАТИ !
